

Patrón de diseño Template Method

El patrón de diseño Template Method (Método Plantilla) es un patrón de **comportamiento** que define el esqueleto de un algoritmo en una clase base, delegando la implementación de ciertos pasos específicos a las subclasses sin permitir que se altere la estructura general del flujo.

Este patrón es ideal cuando múltiples clases ejecutan procesos que comparten la misma estructura lógica, pero difieren en la forma en que realizan determinados pasos individuales.

Componentes Clave

El patrón se estructura a partir de los siguientes elementos:

- **Clase Abstracta (Abstract Class):** Define el "**método plantilla**" que contiene la secuencia del algoritmo, y declara los métodos abstractos (o pasos) que las subclasses deberán implementar.
- **Método Plantilla (Template Method):** Es el método principal (usualmente marcado como definitivo o no modificable) que coordina y ejecuta los pasos del proceso en un orden estricto.
- **Clases Concretas (Concrete Classes):** Subclasses que heredan de la clase abstracta e implementan los pasos específicos del algoritmo según sus propias necesidades.

Ejemplo Práctico (Pseudocódigo)

Imaginá un sistema que procesa diferentes tipos de archivos (como PDF y CSV) para extraer datos. La estructura del proceso siempre es la misma:

- abrir el archivo,
- extraer la información,
- procesar los datos y
- cerrar el archivo.

```
// Clase Base Abstracta
class abstracta ProcesadorArchivos {

    // Este es el Método Plantilla que define el flujo fijo del algoritmo
    funcion procesarDocumento() {
        abrirArchivo()
        extraerDatos()
        analizarDatos()
        cerrarArchivo()
    }

    // Pasos comunes ya implementados
    funcion abrirArchivo() {
        imprimir("Abriendo archivo...")
    }

    funcion cerrarArchivo() {
        imprimir("Cerrando archivo y liberando memoria...")
    }
}
```

```

// Pasos abstractos que cada subclase debe definir
abstracto funcion extraerDatos()
abstracto funcion analizarDatos()
}

// Clase Concreta para PDFs
clase ProcesadorPDF hereda de ProcesadorArchivos {
    funcion extraerDatos() {
        imprimir("Extrayendo texto y gráficos del PDF.")
    }
    funcion analizarDatos() {
        imprimir("Aplicando filtros de análisis para PDF.")
    }
}

// Clase Concreta para CSVs
clase ProcesadorCSV hereda de ProcesadorArchivos {
    funcion extraerDatos() {
        imprimir("Leyendo filas y columnas separadas por comas.")
    }
    funcion analizarDatos() {
        imprimir("Analizando datos tabulares del CSV.")
    }
}

```

Ventajas	Desventajas
Reutilización de código: Evita la duplicación al colocar la lógica común en la superclase.	Rigidez arquitectónica: Algunas subclases pueden verse limitadas por el esqueleto fijo del algoritmo.
Fácil mantenimiento: Si el flujo global cambia, solo se modifica en un único lugar.	Complejidad de extensión: Puede ser difícil de mantener a medida que el algoritmo acumula demasiados pasos intermedios.
Flexibilidad focalizada: Permite a los clientes alterar solo partes específicas de un proceso extenso.	Riesgo de diseño: Existe la posibilidad de violar el Principio de Sustitución de Liskov si una subclase elimina un paso por defecto.